Machine Learning in Python

Last Update: October 30, 2026 — Maximilien Notz

Numpy

np.random

`.seed(seed=S)`	Set the seed to `S` or reset if `S` is none.
`.random_sample(size=s)`	return a np array or `s` random floats between `l` and `h`.
`.randint(l,high=h,size=s)`	return a np array or `s` random integers between `l` and `h`.
`.shuffle(x)`	Shufle the array `x`.
`.permutation(m)`	return the indices of random permutation of a list of `m` elements.

`.choice(a,size=s,replace=True,p=p)` Generates a random sample of size `s` from the 1d array a with probability distributions `p` with replacements or without.

Pytorch

Tensor

Let `t` be a pytorch tensor and the following are from `torch`:

`.randn(s)`	Create a random tensor of rank 1
`.ones(s)`	Create a tensor of ones of rank 1
`.zeros(s)`	Create a tensor of zeros of rank 1
`.cat(lst,dim=d)`	Concatenate the tensors in lst along dimension `d`
`.mul(t1,t2)`	Element-wise multiplication of `t1` and `t2`
`.add(t1,t2)`	Element-wise addition of `t1`
`t.size()`	Return the size of the tensor `t`
`t.view(s)`	Reshape the tensor `t` to the shape

Deep Learning

All the following are in `torch.nn`:

`.linear(m, n)`	Apply a afine linear transformation object with `m` inputs and `n` outputs.
`.conv1d(m, n, k)`	Apply 1D convolution with `m` input chanels and `n` output chanels with a kernel `k`.
`.conv2d(m, n, k)`	Apply 2D convolution with `m` input chanels and `n` output chanels with a kernel `k`.
`.conv3d(m, n, k)`	Apply 3D convolution with `m` input chanels and `n` output chanels with a kernel `k`.

`print(obj,sep='|',end='.',file=sys.stdout,flush=False)` Prints `obj` to the specified `file` with a separator `sep` between objects and an ending character `end` at the end

Activation Function

All `Pytorch` are not performed in place unless we add the argument `inplace=False`.

`.Sigmoid()`	$\sigma(x) = \frac{1}{1+e^{-x}}$
`.ReLU()`	$\text{ReLU}(x) = \max(0, x)$
`.LeakyReLU(negative_slope=m)`	$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ mx & x \leq 0 \end{cases}$
`.Tanh()`	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Regularization

`.Dropout(p=P)`	Dropout regularization with probability `P`
`.BatchNorm()`	Batch normalization

Scikit Learn

K-Fold Cross-Validation

Snippet for K-Fold Cross-Validation using Scikit Learn:

```
kfold = KFold(n_splits=5, shuffle=True)
scores = cross_val_score(model, X, y, cv=kfold,
    scoring='accuracy')

print("Folds accuracy:", scores)
print("Mean Accuracy:", np.mean(scores))
```

Optuna

Optuna is a powerful hyperparameter optimization framework that automates the process of searching for the best hyperparameters for machine learning models.

General Shape

```
def Objective(trial):
    # Define hyperparameters to optimize
    param1 = trial.suggest_float('param1', low, high)
    param2 = trial.suggest_int('param2', low, high)

    # Create and train your model
    model = YourModel(_param1=param1, _param2=param2)
    kfold = KFold(n_splits=5, shuffle=True)
    scores = cross_val_score(model, X, y, cv=kfold,
            scoring='accuracy')

    # Evaluate the model on validation data
    score = model.score(X_val, y_val)

    return score
study = optuna.create_study(direction='maximize')
study.optimize(Objective, n_trials=100)
print("Best hyperparameters:", study.best_params)
fig = optuna.visualization.plot_slice(study,
    params=["param1", "param2"])
show(fig)
```

Pruning

Optuna pruning allows you to stop unpromising trials early, saving computational resources and speeding up the optimization process. You can enable pruning by using `trial.should_prune()` inside the training loop which method to check if the trial should be pruned.

Base Models

XGBoost

LightGBM

ROCKET

MINIROCKET